

A Reproducible CPU-to-GPU Pipeline for Transfer-Entropy Networks: The Econstellar Compute Run behind the Commodity Study

Avishek Bhandari*

June 2026

Abstract

Measuring which markets transmit information to which others is computationally demanding. The model-free estimator we favour searches for nearest neighbours in a reconstructed state space, and establishing that a measured flow is more than noise multiplies that search roughly a hundredfold through surrogate resampling. This note documents how the *Econstellar* compute engine carried out that calculation for a companion study of twenty-three commodity futures across eight crisis windows, and what we learned by moving the heaviest step from twenty-two CPU cores onto a single graphics processor. Three points are worth recording for anyone building similar pipelines. First, the migration is safe only because it is verified: the GPU’s transfer-entropy estimates are bit-for-bit identical to the CPU reference—agreeing to within 9×10^{-16} across all 506 ordered pairs of a window—so no published number can move when the compute path changes. Second, the GPU is not a free lunch: for the bare estimate the twenty-two CPU cores are faster, and the GPU only repays its fixed overhead once surrogate testing inflates the work, with the margin growing as the sample lengthens. Third, two of our hardest bugs were not performance problems but correctness ones—a just-in-time compiler with no on-disk cache, and a resampling scheme that silently breaks a nearest-neighbour estimator—and both are reusable cautions. We argue that for this class of empirical work the computation is part of the scientific claim, and that documenting it, with its honest limits, is part of reproducibility.

Keywords: Reproducible computation; Transfer entropy; GPU acceleration; Nearest-neighbour estimation; Surrogate testing; Commodity markets

JEL Codes: C58; C63; C88; G01

1 Why the computation is part of the claim

A companion paper asks two questions of twenty-three commodity futures over 1991–2025: which commodities move together over long horizons, and which lead the others (Bhandari, 2026b). The first question is answered with a symmetric long-memory object—the Achard–Gannaz fractal-connectivity network (Achard and Gannaz, 2016; Achard et al., 2008)—and the second with a directed transfer-entropy network estimated by the nearest-neighbour scheme of Kraskov et al. (2004) in the conditional-mutual-information form of Frenzel and Pompe (2007). The economics is reported there. Here we report the computation.

*School of Humanities, Social Sciences and Management, Indian Institute of Technology Bhubaneswar, India. Email: avishekb@iitbbs.ac.in. This is a compute-run companion to Bhandari (2026b); it documents the computation rather than re-deriving the economics. Code for every figure and timing in this note is archived with the manuscript; see the data and code statement.

The two are not cleanly separable. What an empirical economist can credibly claim is bounded by what can be computed: whether significance can be assessed against a proper nonlinear null, whether the estimator’s sensitivity to its tuning can be swept, whether confidence bands can be drawn without a method that quietly invalidates the estimator. Each of these is a compute decision before it is an econometric one, and each shaped the findings of the companion study. We therefore treat the compute run as a first-class artefact, document the platform that produced it, and are deliberately honest about where hardware acceleration helped and where it did not. The numbers in this note are themselves reproducible: the benchmark scripts are archived with the paper and were re-run to produce every timing reported below.

2 The econometric workload

What is computed. The pipeline has two layers (Figure 1). The *symmetric* layer estimates the multivariate wavelet-Whittle fractal-connectivity matrix and decomposes it into communities by the fast-greedy algorithm of Clauset et al. (2004); it is inexpensive and runs on the CPU. The *directional* layer is the bottleneck. For an ordered pair of return series ($Y \rightarrow X$) transfer entropy is the conditional mutual information

$$T_{Y \rightarrow X} = I\left(X_{t+1}; Y_t^{(\ell)} \mid X_t^{(\ell)}\right), \quad (1)$$

the reduction in uncertainty about X ’s next value once the recent past of Y is known, beyond what X ’s own past already explains. We estimate (1) directly from data with the k -nearest-neighbour estimator ($k = 4$, embedding $\ell = 1$, maximum norm), which assumes no functional form and so captures the nonlinear and tail dependence that linear Granger causality misses. Crucially, it is computed on raw log-returns, which are stationary (confirmed by ADF and KPSS), never on price levels.

Why it is heavy. A bare estimate of (1) for one pair already requires a neighbour search that is quadratic in the window length m . But a bare estimate is not enough: the estimator is biased away from zero in finite samples, so a measured flow means nothing until it is compared with a null that destroys directional structure while preserving each series’ own distribution and autocorrelation. We build that null with $B = 99$ iterative amplitude-adjusted Fourier-transform (IAAFT) surrogates of the source (Schreiber and Schmitz, 1996; Schreiber, 2000), and report the *effective* transfer entropy—the estimate minus its surrogate-mean bias—following Marschinski and Kantz (2002). The cost of one window is therefore

$$\underbrace{N(N-1)}_{506 \text{ ordered pairs}} \times \underbrace{(B+1)}_{100 \text{ searches}} \times \underbrace{\mathcal{O}(m^2)}_{\text{neighbour search}} = 50,600 \text{ neighbour searches per window}, \quad (2)$$

with $N = 23$. The surrogate factor of one hundred, not the pair count, is what turns a quick calculation into a heavy one—and, as Section 4 shows, it is exactly the factor that makes a GPU worthwhile.

3 The Econstellar engine and the CPU-to-GPU dispatch

The calculation ran on *Econstellar*, a reproducible computational-economics engine that exposes a registry of more than twenty parameterised econometric methods behind a single interface. “Parameterised” is the security and reproducibility guarantee: a caller selects a named method and supplies typed arguments, never arbitrary code, so the surface that can be executed is fixed and auditable. Transfer entropy is one such method; the daily systemic-risk index, built originally for an equity panel and reused here without modification, is another.

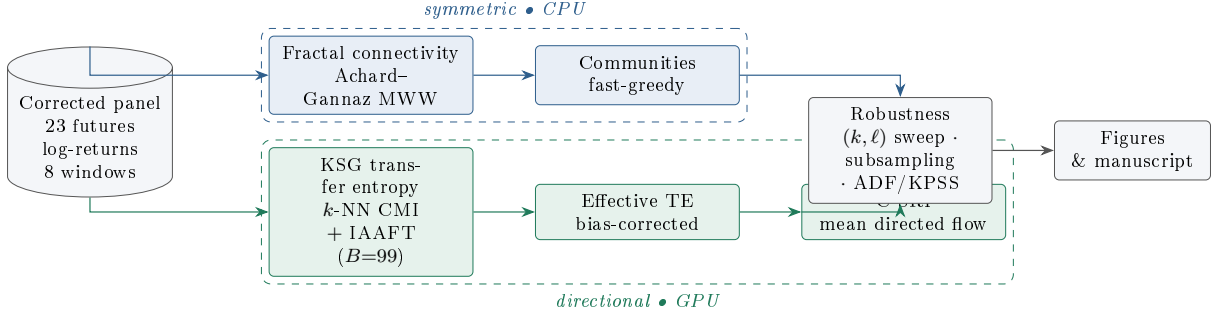


Figure 1: **The two-layer pipeline.** A symmetric long-memory layer (blue, CPU) and a directional transfer-entropy layer (green, GPU) are estimated on the same per-window samples, combined through a shared robustness battery, and emitted as the figures and tables of the companion study. The green layer is the computational bottleneck.

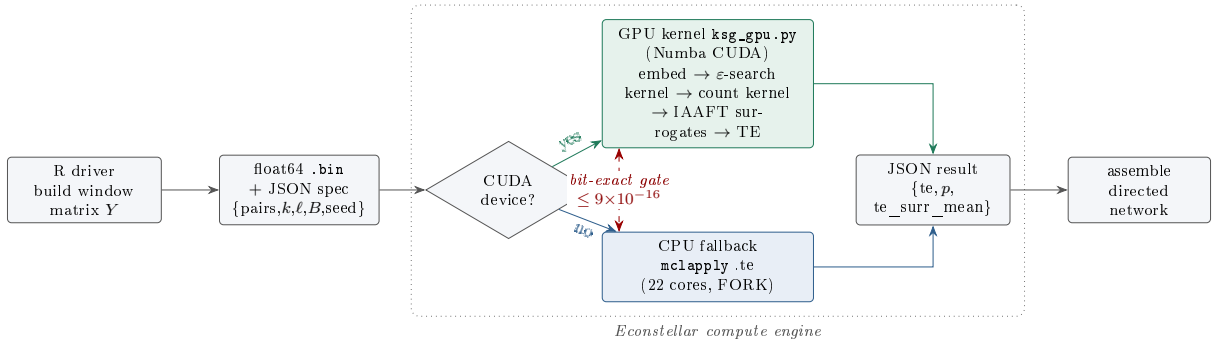


Figure 2: **Automatic, verified dispatch.** The driver serialises the window and probes for a GPU. The CUDA kernel and the CPU reference compute the same transfer entropy; a standing equivalence gate holds the two to within 9×10^{-16} , so offloading never moves a number. Missing or broken hardware degrades silently to the CPU path.

The estimator exists in two implementations that are required to agree (Figure 2). The reference is a transparent R routine (`_ksg_core.R`) that any reviewer can read. The accelerated path is a CUDA kernel (`ksg_gpu.py`, written with Numba (Lam et al., 2015)) that performs the same neighbour search and the same IAAFT construction on the GPU. Dispatch is automatic and defensive: the R driver writes the window matrix as a column-major binary blob and a small JSON specification (the pairs, k , ℓ , B and the surrogate seed), probes for a usable CUDA device, and offloads if one is present; if the probe fails—no device, a broken toolkit, or an operator override—it falls back silently to a governed CPU path that parallelises the pairs across all cores with a FORK cluster. A box with no GPU still produces the identical answer, only more slowly. This is the sense in which the platform is adaptive: it senses its hardware and routes the workload accordingly, and a self-checking equivalence gate (Section 4) guarantees the route never changes a result.

4 What the migration bought

Fidelity first. Acceleration is worthless if it changes the answer, so the equivalence check is the gate the migration had to pass. The observed transfer entropy returned by the GPU kernel matches the CPU reference to floating-point rounding: across all 506 ordered pairs in each of the three windows benchmarked here the maximum absolute difference is 8.9×10^{-16} , and the companion study’s per-window check, spanning all eight windows, agrees to 4.4×10^{-16} —both indistinguishable from zero at any reported precision. Because the surrogate draw is seeded, the

Table 1: **Wall-clock time, one window, 22 CPU cores vs. one GPU.** 506 ordered pairs, $k = 4$, $\ell = 1$. $B = 0$ is the bare observed estimate; $B = 99$ is the production setting with IAAFT surrogate testing (50,600 neighbour searches). Observed-estimate agreement (bit-exactness) is reported in the text. A speed-up below one means the CPU is faster.

Window	m	$B = 0$ (observed only)		$B = 99$ (production)		
		CPU (s)	GPU (s)	CPU (s)	GPU (s)	speed-up
Stock Crash 2015–16	337	0.73	3.40	98.9	76.0	$1.3\times$
COVID-19	501	1.32	3.82	254.8	176.0	$1.4\times$
Pre-COVID-19	751	2.07	4.05	541.2	263.5	$2.1\times$
<i>speed-up</i> ($B = 0$)		0.2–0.5 $\times$ (CPU wins)				

permutation p -values are reproducible as well; the transfer-entropy magnitudes are deterministic and seed-independent. The practical consequence is that the choice between CPU and GPU is a choice about *speed alone*, and the engine is free to make it automatically.

Where the GPU helps, and where it does not. Table 1 reports wall-clock time for one window on the twenty-two CPU cores of the workstation against a single NVIDIA RTX 3000 Ada GPU, for two workloads: the bare observed estimate ($B = 0$) and the production setting with surrogate testing ($B = 99$). The result is instructive and easily misread. For the bare estimate the CPU is several times *faster*: 506 small neighbour searches do not amortise the GPU’s fixed costs—a one-off just-in-time kernel compilation of roughly three seconds, plus per-launch and host-to-device transfer overhead. Only when surrogate testing multiplies the work a hundredfold does the GPU move ahead, and the margin widens with the window length, because longer windows make each neighbour search more expensive and so push the balance further from fixed overhead towards parallelisable arithmetic (Figure 3). The speed-up is real but moderate—the honest figure is a small multiple, not the hundredfold that “GPU acceleration” sometimes connotes—and it should be: the CPU baseline is already twenty-two cores, so this is a fair contest of twenty-two mature cores against one laptop-class accelerator. Two further benefits do not appear in the table. Offloading the search frees those twenty-two cores for the concurrent symmetric layer, and it lifts a load that otherwise saturates the machine. Over the full study—eight windows, the (k, ℓ) robustness sweep and the subsampling bands—the GPU path completes the headline networks in about twenty minutes.

5 Three compute lessons

The two slowest parts of this project to get right were not about speed. They were about correctness, and both are general enough to record.

1. A just-in-time kernel has no on-disk cache. The Numba CUDA kernel compiles on first use—the roughly three-second floor in the $B = 0$ GPU column of Table 1 is that compilation, not computation. For a single run this is paid once and forgotten. But the subsampling confidence bands require recomputing the network on hundreds of resamples, and the natural implementation—relaunch the kernel per resample—re-compiles every time, turning a half-hour job into an overnight one. The fix is to keep one Python process alive per window and loop the resamples inside it, so the kernel is compiled once and the bias correction (the expensive $B = 99$ step) is computed once and held fixed while the cheap resampled estimates are drawn. The lesson generalises to any JIT-accelerated inner loop: amortise the compile, do not pay it per iteration.

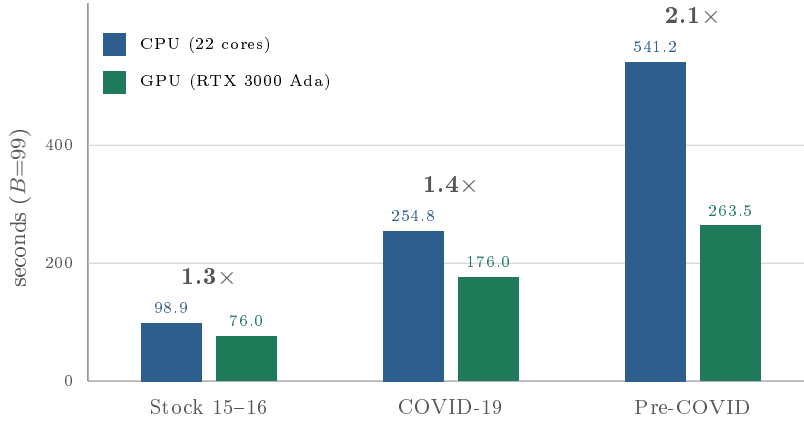


Figure 3: **Production-setting wall time ($B = 99$) by window.** The GPU’s advantage over the twenty-two-core CPU grows with window length m , because longer windows make the neighbour search dominate the GPU’s fixed overhead. For the bare estimate ($B = 0$, Table 1) the ordering reverses and the CPU is faster.

2. A bootstrap silently breaks a nearest-neighbour estimator. The obvious way to put a confidence band on a window-level statistic is a block bootstrap—resample blocks of days with replacement. For a k -nearest-neighbour estimator this is not merely inefficient, it is *invalid*. Resampling with replacement duplicates rows; duplicated points sit at distance zero in the embedding; the neighbour search then finds its k neighbours at radius zero and the estimator diverges. In our data the symptom was unmistakable—a bootstrap mean transfer entropy of 0.234 against a point estimate of 0.002, an inflation of two orders of magnitude. The remedy is block *subsampling without replacement* (Politis et al., 1999): contiguous blocks of a fixed common size, every row distinct, the estimator never seeing a degenerate point. We mention this because the failure is silent—the bootstrap returns plausible-looking numbers—and because it is a property of the estimator, not of these data, so it will recur for anyone combining resampling with k -NN information measures.

3. The accelerator’s value lives in the surrogate dimension. Because the GPU only repays its overhead under surrogate testing (Section 4), the kernel is designed around that dimension: the $B + 1$ neighbour searches for a pair—the observed series and its B surrogates—are batched into a single device launch rather than issued one at a time. This is the architectural expression of (2): the factor of one hundred is not a cost to be endured but the parallelism that justifies the hardware.

6 An accreting, self-verifying platform

The features that made this run trustworthy were not built for it. The transfer-entropy method, the IAAFT surrogate machinery, and the mean-flow systemic-risk index were all written for an earlier study of G20 equities and were reused here unchanged; the commodity application added data and a second (fractal-connectivity) layer, not new estimation code. This accretion is deliberate: each paper leaves behind a verified, parameterised method that the next can call, and the registry has grown across studies rather than being rebuilt for each. What keeps accretion safe is a standing eval suite that re-checks every gated method—the CPU–GPU equivalence gate of Section 4 is one such check—so that a change made for one paper cannot silently regress another. The results are published through a portal that renders each compute run and lets a reader re-execute the headline methods, holding the platform to the rule that only a genuine re-run earns the label “verified.” We make no claim that this amounts to an autonomous or self-

improving intelligence; it is something more modest and, for research, more useful—a compute layer that remembers what it has been taught, verifies itself against a fixed reference, and adapts its execution to the hardware it finds.

7 Conclusion

For information-theoretic network econometrics the computation is not a back-office detail; it determines which claims are feasible and how far they can be trusted. We have set out, for one study, exactly how the estimate was produced, why the GPU helps only past a threshold the surrogate test sets, and what two silent failures taught us. The honest summary is unglamorous: a verified, hardware-adaptive pipeline that is bit-for-bit reproducible on a laptop and a few times faster when the statistics demand it. That it is unglamorous is the point—reproducibility is made of such details, and they are worth writing down.

Acknowledgements

The computation reported here was carried out on the *Econstellar* compute engine, the reproducible, GPU-accelerated computational-economics node developed by the author. The work was supported by a Google Cloud Platform research grant and by a seed grant of the Indian Institute of Technology Bhubaneswar (grant code SP128). The author thanks the School of Humanities, Social Sciences and Management at IIT Bhubaneswar for institutional support. The GPU node is a single NVIDIA RTX 3000 Ada device; no cloud compute was billed for the runs in this note.

Data and code availability

The benchmark scripts that produced Table 1 and Figure 3 (`bench_cpu_gpu.R` for the bit-exactness and $B = 0$ timing, `bench_b99.R` for the production setting), the CPU reference estimator, and the GPU kernel are archived with the manuscript and with the companion study (Bhandari, 2026b). Every timing in this note was produced by re-running those scripts; transfer-entropy magnitudes are deterministic and the surrogate draw is seeded.

References

- Achard, S., Bassett, D. S., Meyer-Lindenberg, A., and Bullmore, E. (2008). Fractal connectivity of long-memory networks. *Physical Review E*, 77(3):036104.
- Achard, S. and Gannaz, I. (2016). Multivariate wavelet Whittle estimation in long-range dependence. *Journal of Time Series Analysis*, 37(4):476–512.
- Bhandari, A. (2026a). Community networks of commodity markets: Long memory models and the impact of different crises. Working paper, Indian Institute of Technology Bhubaneswar.
- Bhandari, A. (2026b). Fractal communities and transfer-entropy networks in commodity markets: Directional information flow and systemic risk across crises. Working paper, Indian Institute of Technology Bhubaneswar.
- Clauset, A., Newman, M. E. J., and Moore, C. (2004). Finding community structure in very large networks. *Physical Review E*, 70(6):066111.
- Frenzel, S. and Pompe, B. (2007). Partial mutual information for coupling analysis of multivariate time series. *Physical Review Letters*, 99(20):204101.

- Kraskov, A., Stögbauer, H., and Grassberger, P. (2004). Estimating mutual information. *Physical Review E*, 69(6):066138.
- Lam, S. K., Pitrou, A., and Seibert, S. (2015). Numba: A LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*. ACM.
- Marschinski, R. and Kantz, H. (2002). Analysing the information flow between financial time series. *The European Physical Journal B*, 30(2):275–281.
- Politis, D. N., Romano, J. P., and Wolf, M. (1999). *Subsampling*. Springer, New York.
- Schreiber, T. and Schmitz, A. (1996). Improved surrogate data for nonlinearity tests. *Physical Review Letters*, 77(4):635–638.
- Schreiber, T. (2000). Measuring information transfer. *Physical Review Letters*, 85(2):461–464.